

Hexagon Media Service

Building, installing and using the app, its engine and the browser extension.
Revised 2026-08-17.

What this is

A private desktop downloader for YouTube, Reddit, Twitter/X, HDRezka and roughly 1800 other sites. Nothing is uploaded, no account is needed, and every setting stays on your machine.

Downloads, screenshots and the short version live at simplefoxofficial.github.io/Hexagon-Media-Service. This manual is the long version.

It is three pieces that ship together:

Piece	Technology	What it does
Shell	Electron	The window and interface
Engine	Python	yt-dlp, tagging, file sorting
Extension	Chrome MV3	Reads HDRezka pages in your browser

The shell talks to the engine over a private pipe, and the engine talks to the extension over a token-protected port on 127.0.0.1. Only the interface is web technology; the downloading is all Python.

1. Installing the released build

If you only want to use the app, this is the whole of it. Download the installer from the releases page and run it. The download engine and ffmpeg are both bundled, so Python is not needed and nothing else has to be installed first.

A portable build is published beside it. It cannot update itself, because a running executable cannot overwrite its own file on Windows, so it links to the release page instead.

HDRezka is the one feature that needs more: the browser extension in section 6. Everything else works immediately.

Note. The rest of this manual is for building from source. Skip to section 6 if you installed the released build.

2. Building: what you need first

Requirement	Version	Why
Python	3.10 or newer	Runs the download engine
Node.js and npm	18 or newer	Runs and packages the shell

ffmpeg	any recent	Merges video with audio, converts audio. Bundled into the packaged build; needed on PATH when running from source
Google Chrome or Edge	any recent	Only needed for HDRezka

Check what you already have:

```
python --version
node --version
ffmpeg -version
```

Note. If python opens the Microsoft Store, install Python from python.org instead and tick **Add python.exe to PATH**. The Store build is sandboxed and PyInstaller cannot always read from it.

3. Installing the dependencies

Engine dependencies

```
cd "D:\GitHub Repos\MediaDownloaderProject"
python -m pip install -r requirements.txt
```

Shell dependencies

```
cd desktop
npm install
```

Note. npm 11 and newer block install scripts, so Electron's runtime download is skipped and starting it fails with *Electron failed to install correctly*. Fix it with `npm approve-scripts electron` followed by a reinstall. If that still fails, download `electron-v<version>-win32-x64.zip` from the Electron releases page, extract it into `node_modules/electron/dist/`, and put the text `electron.exe` into `node_modules/electron/path.txt`.

4. Running from source

This is the normal way to use it during development:

```
cd desktop
npm start
```

The window opens and the engine starts automatically as a child process. You never launch the engine yourself. Add `-- --dev` to open developer tools.

5. Building executables

There are two builds. The engine is frozen with PyInstaller, then the shell is packaged with electron-builder and the engine is bundled inside it.

Step one: freeze the engine

```
cd "D:\GitHub Repos\MediaDownloaderProject"
python -m PyInstaller Engine.spec --noconfirm ^
--distpath dist-engine --workpath build-engine
```

Produces `dist-engine\mediadl-engine.exe`. It is a console-less program that speaks JSON over its input and output, so double-clicking it does nothing visible. That is expected.

Step two: package the shell

```
cd desktop
npm run dist
```

Produces an installer, a portable build and `latest.yml` in `build-output\`. electron-builder copies `dist-engine` and `vendor\ffmpeg` into the app's resources, and the shell prefers that frozen engine over your Python install when it finds it.

If `vendor\ffmpeg` is empty, run `python tools\stage_ffmpeg.py` first. Without it the app ships without `ffmpeg` and can neither merge video with audio nor convert audio.

Publishing a release

Bump version in `desktop\package.json` and `__version__` in `mediadl\__init__.py`, build both steps, then publish a GitHub release tagged `v<version>` with all three files attached: the Setup exe, the portable exe and `latest.yml`.

Note. `latest.yml` carries the checksum the updater verifies a download against. Leave it off the release and updates still install, but only the file size was ever checked.

Optional: the icon

```
python tools\make_icon.py
```

Regenerates `mediadl\resources\app.ico` from the logo at seven sizes. Only needed if you change the mark.

Build reference

Command	Output
<code>python -m PyInstaller Engine.spec</code>	<code>dist-engine\mediadl-engine.exe</code>
<code>npm run dist</code>	<code>build-output\</code> installer, portable, <code>latest.yml</code>
<code>python tools\stage_ffmpeg.py</code>	<code>vendor\ffmpeg\</code>
<code>python tools\make_icon.py</code>	<code>mediadl\resources\app.ico</code>

Note. A running copy locks its own executable, so a rebuild fails with a permission error that looks unrelated. Close the app first. The PowerShell build script does this for you.

6. Installing the browser extension

Only needed for HDRezka. The site refuses requests that do not come from a real browser, so the page is read inside yours; the extension is the bridge between that page and the app.

The same five steps are on the website with a picture for each, in English, Russian and Ukrainian: simplefoxofficial.github.io/Hexagon-Media-Service/extension.html.

1. Open `chrome://extensions` and turn on **Developer mode** (top right).
2. Click **Load unpacked** and choose the extension folder in the project.
3. Start the app, then go to **Download** → **HDRezka**. It shows an address and a pairing token.
4. Click **Copy token**, then open the extension's **Options**, paste it, and press **Save and test**.
5. You should see *Connected*. The app's HDRezka tab turns green too.

Note. The token is what stops any web page you visit from talking to the app. It is generated once and kept in your settings folder. The extension only ever contacts 127.0.0.1, never the internet.

7. Using it

Downloading from YouTube and most sites

1. Go to the **Download** tab and pick **Auto detect** or **YouTube**.
2. Paste one or more links, one per line. Playlists and channels expand into separate queue items on their own.
3. Choose the mode, quality and container.
4. Set the destination folder, then press **Add to queue**.

The view switches to **Queue**, where each item shows progress, speed and remaining time, with pause, resume, retry and reveal-in-folder controls.

The window

The app draws its own titlebar, so the window controls sit at the right of the top bar rather than in a separate strip. Drag the empty area to move the window, and double-click it to maximise. **F11** toggles fullscreen and **Esc** leaves it; the middle button changes to an inward-arrow icon while fullscreen, and pressing it returns to a normal window.

Downloading from HDRezka

1. Open the film or series page in your browser and leave the tab open.
2. In the app, go to **Download** → **HDRezka** and press **Read open HDRezka tab**.
3. Pick the translation (dub) and quality. Both come from the page itself.
4. Tick what you want in the season tree. Each season expands to its episodes, and the season checkbox selects the whole season.
5. Press **Download**. Progress appears in the **Queue** tab.

Selection shortcuts in the tree:

- **Select all** ticks every episode of every season.
- **Clear** unticks everything.
- The range box accepts 1-10 or 3, 5, 7, and applies to whichever seasons are expanded, or all of them if none are.
- A part-selected season shows a dash rather than a tick, with a 3 / 21 count beside it.

Note. HDRezka files are fetched by the browser, so they land in your Downloads folder first and the app moves them to the real destination when each one finishes. Chrome will not let an extension write anywhere else. Nothing is left behind; the staging folder is a waypoint.

Size checks and batching

Before anything starts, one episode is measured and multiplied by how many you selected. The figure is approximate but close, because episodes of a series are similar sizes.

That total is compared against free space on the drive the browser downloads to. If a whole season would not fit, the work is split into batches automatically: a batch downloads, those files are moved to the destination, and only then does the next batch start. The staging drive therefore never holds more than one batch at a time. The Queue heading shows which batch is running and how many of its files have been filed.

What the queue is telling you

A browser download passes through several stages, and each one is named in the queue so a slow step is never mistaken for a broken one:

Stage	Meaning
Downloading in browser	Chrome is fetching the file. Progress is real.
Waiting for the browser to release the file	The transfer finished but Chrome still holds the file. The app is checking it has stopped growing and matches the expected size.
Moving to destination	Copying from the Downloads folder to your chosen folder. Progress is real. A large file across drives takes a while; this is normal.
Writing tags	Adding title, show, season and episode metadata.
Completed	The file is in place and verified.

Note. A file is only ever deleted from the staging folder after the copy has been verified byte for byte. If a move fails the original is left untouched, so nothing is lost. A half-written file is never left where a finished one should be.

Where files end up

Series are filed by show and season:

```
D:\Movies\House M.D\  
House M.D.\  
Season 06\  
House M.D. 6x20 LostFilm.mp4  
House M.D. 6x21 LostFilm.mp4
```

The name pattern is show season x episode dub. The dub is dropped when unknown, and the show prefers the page's original title when it has one. Change the pattern in **Settings**; a broken pattern falls back to a safe default rather than failing.

The Settings panel

Setting	What it does
Theme and accent	Dark or light, and one of five accent colours.
Simultaneous downloads	How many run at once. Three is a sensible default.
Speed limit	Kilobytes per second across all downloads. 0 is unlimited.
Expand playlists	Off means a playlist becomes one item, not many.

Season folders	Turn off to keep every episode in one flat folder.
Embed metadata	Titles, chapters and cover art written into the file.
Use cookies from	For age-restricted or members-only videos.
Updates	See section 8.

The engine understands a good deal more than the panel exposes: audio codec and bitrate, per-site sub-folders, the filename and episode-name templates, subtitle languages, a proxy, retry counts, chunked downloads, strict container matching and a download archive. Those are read from `settings.json` at startup - see section 9 for where it lives.

Note. Two of them are worth leaving alone. **Chunked downloads** stops long transfers dying when a media URL expires, and **strict container matching** is a known cause of HTTP 403 partway through a YouTube download. The defaults are the safe ones.

8. Updates

The app checks its own GitHub releases for a newer version. The check is one request when the window opens, and it is the only thing the app contacts on its own behalf. Nothing is downloaded until you ask for it.

What you see

- A green **Update** pill appears in the top bar when a newer release exists. It is absent the rest of the time.
- **Settings** → **Updates** shows the installed version, when it last checked, and a **Check now** button.
- Either one opens a window with the release notes and, on the installer build, a **Download and install** button.
- After an update installs, the notes for the new version are shown once, automatically. **View changelog** brings them back at any time.

What happens when you install one

1. The installer for the new version is downloaded to a temporary folder, with progress shown in the window.
2. It is checked against the size and the SHA-512 checksum published beside it in the release. A file that does not match is discarded and nothing is installed.
3. The installer runs silently, closing the app as it starts.
4. The app reopens on the new version. Settings, history and downloads are kept.

Note. Only the installer build can replace itself. The portable build links to the release page instead, because a running executable cannot overwrite its own file on Windows. Running from source disables installing entirely.

Turning it off

Check for updates when the app starts in Settings. With it off, nothing is contacted until you press **Check now** yourself. The choice, the last check time and the last few releases seen are kept in `updates.json` beside your settings, which is what lets the changelog still open when GitHub is unreachable.

9. When something goes wrong

The **Logs** tab shows what the app and yt-dlp actually did. **Copy** puts the whole buffer on the clipboard and **Log folder** opens the directory holding the rotating file:

```
%APPDATA%\HexagonMediaService\mediadl.log
```

Common problems

Symptom	Cause and fix
<i>Requested format is not available</i> on a video that plays fine	yt-dlp is out of date. Run <code>python -m pip install --upgrade yt-dlp</code> and rebuild the engine.
Every download fails right after enabling cookies	Chrome locks its cookie database while running. Close Chrome, or set Use cookies from back to None. The app warns and carries on without them.
HTTP 403 partway through a large download	The media URL expired. It retries automatically. If you have edited settings.json, leave <code>http_chunk_size_mb</code> non-zero and <code>strict_container_match</code> false.
<i>ffmpeg: missing</i> in Settings → About	Only possible when running from source: install ffmpeg and put it on PATH, or set <code>ffmpeg_path</code> in settings.json. The packaged build carries its own copy in <code>resources\ffmpeg</code> .
<i>Could not establish connection</i> from the extension	The content script is not in that tab. Reload the extension at <code>chrome://extensions</code> , then reload the HDRezka tab once.
<i>This does not look like a HDRezka title page</i>	You are on a listing or search page rather than a title, or the site changed its markup.
HDRezka returns an anti-bot page	Open the title in your browser and let its check pass, then use the extension. The app cannot get past it on its own, by design.
Nothing downloads and the engine dot is red	The engine did not start. In a released build that is a bug worth reporting with the log. From source, check Python is on PATH and the requirements are installed.
A download sits on <i>Moving to destination</i> for a long time	It is copying between drives, which is a real copy rather than a rename. The percentage moves; leave it. Nothing is deleted until the copy is verified.
<i>Size mismatch</i> in the log and the file was not filed	The browser transfer ended early. The partial file is deliberately left in the staging folder rather than filed as if it were complete. Download that episode again.

Where things live

What	Where
------	-------

Settings	%APPDATA%\HexagonMediaService\settings.json
History	%APPDATA%\HexagonMediaService\history.json
Update state and changelog	%APPDATA%\HexagonMediaService\updates.json
Pairing token	%APPDATA%\HexagonMediaService\bridge-token.txt
Log file	%APPDATA%\HexagonMediaService\mediadl.log
Browser staging	Downloads\MediaDownloader\

The folder is migrated from the older MediaDownloader name the first time this version runs, so an upgrade keeps its settings, history and pairing token. The browser staging folder still uses the old name, because it is Chrome that writes there.

10. Keeping it working

Sites change their players constantly, so yt-dlp goes stale faster than anything else here. When downloads that used to work start failing, update it first:

```
python -m pip install --upgrade yt-dlp
```

Then rebuild the engine so the packaged app picks it up. The version in use is shown in the app's **Settings** panel under About, and on the right of the Download page.

If you are using a released build rather than your own, this is what the update in section 8 is for: a new release carries a fresher yt-dlp.

Download only what you have the right to download.